Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

Тема: «Бинарное дерево поиска»

Выполнили:
студенты гр. 25ВВВ1:
Ячменев М. А.
Аверин А. А.

Приняли:
к.т.н., доцент Юрова О. В.
ст. преподаватель Деев М. В.

Пенза 2026

1. Цель работы

Изучение принципов организации и функционирования древовидных динамических структур данных. Исследование алгоритмов построения бинарного дерева поиска (БДП), алгоритмов поиска узлов по заданному ключу, подсчета числа вхождений элементов, а также вставки с исключением дубликатов. Аналитическая и экспериментальная оценка вычислительной сложности поиска в сбалансированном и вырожденном деревьях. Приобретение практических навыков разработки программ на языке Си в среде Microsoft Visual Studio с динамическим управлением памятью (malloc, free).

2. Лабораторное задание

В соответствии с методическими указаниями к лабораторной работе № 4 требуется выполнить следующие задания:

- 1. Изучить базовую реализацию алгоритма формирования бинарного дерева поиска с использованием рекурсивных процедур и функции отображения дерева на экране с поворотом на 90 градусов (print_tree).
- 2. Задание 1: Разработать и реализовать функцию поиска узла в бинарном дереве поиска по заданному целочисленному значению ключа (search_tree) с подсчетом фактического числа выполненных сравнений.
- 3. Задание 2: Разработать алгоритм и программную функцию подсчета количества вхождений заданного значения ключа в бинарном дереве (count_occurrences).
- 4. Задание 3 (*): Реализовать функцию добавления элементов в дерево, исключающую запись дубликатов (insert_unique), обеспечивая корректное хранение множества уникальных ключей.
- 5. Задание 4 (*): Провести аналитическую и экспериментальную оценку временной сложности алгоритма поиска: для лучшего случая (корень, O(1)), среднего случая (сбалансированное дерево, O(log2 N)) и худшего случая (вырожденное в линейный список дерево, O(N)).
- 6. Разработать интерактивное меню пользователя и демонстрационный модуль в среде Microsoft Visual Studio на языке Си, провести комплексное тестирование и оформить отчет.

3. Теоретические сведения

3.1. Определение и свойства бинарного дерева поиска (БДП)

Дерево — нелинейная связная иерархическая динамическая структура данных, состоящая из узлов (вершин) и направленных связей (ребер). Бинарным деревом называется дерево, у которого каждая вершина имеет не более двух непосредственных потомков, традиционно называемых левым и правым поддеревьями.

Бинарным деревом поиска (Binary Search Tree, BST) называется бинарное дерево, обладающее характеристическим свойством упорядоченности:

- Для каждого узла X все ключи в его левом поддереве строго меньше ключа узла X (Key(left) < Key(X)).
- Все ключи в его правом поддереве больше либо равны ключу узла X (Key(right) >= Key(X)).
- Левое и правое поддеревья каждого узла также являются бинарными деревьями поиска.

Каждый узел бинарного дерева в динамической памяти представляется структурой, содержащей информационную часть и два адресных поля-указателя:

```
struct Node {
    int key;               // информационное поле (ключ узла)
    struct Node *left;     // указатель на левое поддерево
    struct Node *right;    // указатель на правое поддерево
};
```

3.2. Аналитическая оценка сложности алгоритмов в бинарных деревьях поиска

Временная сложность операций поиска, вставки и удаления в бинарном дереве поиска напрямую зависит от высоты дерева H (максимальной длины пути от корня до листа):

- Лучший случай: искомый элемент находится непосредственно в корне дерева. Требуется ровно 1 операция сравнения: T_best(N) = O(1).
- Средний случай (сбалансированное дерево): при случайном порядке поступления ключей высота дерева составляет H ≈ log2(N). При каждом шаге сравнения алгоритм отсекает примерно половину оставшихся узлов, обеспечивая логарифмическую сложность: T_avg(N) = O(log2 N).
- Худший случай (вырожденное дерево): если элементы поступают в строго отсортированном порядке (по возрастанию или убыванию), бинарное дерево

вырождается в односвязный линейный список («лозу»). Высота такого дерева равна числу элементов (H = N), а сложность поиска падает до линейной: T_worst(N) = O(N).

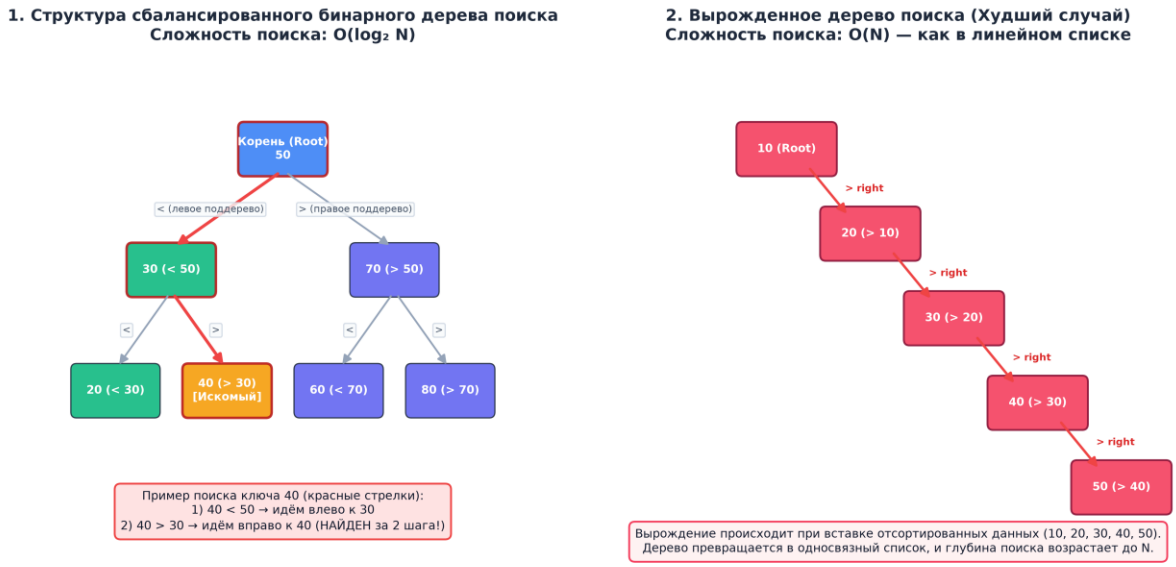


Рисунок 1 – Структурная организация бинарного дерева поиска: сбалансированное дерево со сложностью поиска O(log2 N) и вырожденное дерево со сложностью O(N)

4. Описание программной реализации

Программный комплекс разработан на языке Си в среде Microsoft Visual Studio. В программе реализованы следующие ключевые функции:

- create_node(int key): выделяет память под новый узел с помощью malloc(), инициализирует информационное поле и обнуляет указатели left и right.
- insert_node(struct Node *root, int key): рекурсивно выполняет поиск позиции и вставку нового ключа в дерево поиска (с допущением дубликатов в правое поддерево).
- insert_unique(struct Node *root, int key, int *added): выполняет вставку ключа с контролем уникальности (Задание 3*). Если ключ уже присутствует в дереве, операция отклоняется, а флаг added сбрасывается в 0.
- search_tree(struct Node *root, int key, int *comparisons): выполняет поиск узла по ключу с инкрементом счетчика сравнений (Задание 1). При key < root->key рекурсивно переходит влево, иначе вправо.
- count_occurrences(struct Node *root, int key): рекурсивно подсчитывает количество вхождений заданного значения в дереве (Задание 2).

- calc_height(struct Node *root): вычисляет фактическую высоту дерева как max(H_left, H_right) + 1.
- count_nodes(struct Node *root): определяет общее количество вершин в дереве.
- print_tree(struct Node *root, int level): наглядно отображает древовидную структуру на экране в текстовом виде с поворотом на 90 градусов (корень слева, правое поддерево вверху, левое — внизу) с пропорциональными отступами.
- free_tree(struct Node *root): безопасно освобождает память всех узлов дерева, гарантируя отсутствие утечек оперативной памяти.
- run_demo_tests(): модуль автоматического комплексного тестирования всех этапов работы.

Таблица 1 – Сравнительная характеристика вычислительной сложности операций в бинарном дереве поиска

Операция в БДП	Лучший случай	Средний случай	Худший случай	Ограничивающий фактор / Условие
Поиск ключа (Search)	O(1)	O(log2 N)	O(N)	Высота дерева H; лучший: корень; худший: вырождение
Вставка ключа (Insert)	O(1)	O(log2 N)	O(N)	Поиск позиции листа для размещения нового узла
Вставка без дубликатов	O(1)	O(log2 N)	O(N)	Проверка наличия существующего узла перед вставкой
Подсчет вхождений	O(log2 N)	O(log2 N + K)	O(N)	K — число дубликатов (локализованы в правом поддереве)
Освобождение памяти	O(N)	O(N)	O(N)	Рекурсивное освобождение памяти каждого узла с вызовом free()

5. Результаты работы программы

Для подтверждения корректности функционирования всех разработанных алгоритмов было выполнено комплексное демонстрационное тестирование

программы в среде Microsoft Visual Studio. Ниже приведен протокол вывода в консоль:

```
============================================================
   ДЕМОНСТРАЦИЯ ВСЕХ ЗАДАНИЙ ЛАБОРАТОРНОЙ РАБОТЫ № 4
============================================================


[ЭТАП 1]: Построение дерева тестовым набором данных...
Исходная последовательность чисел:
50 30 70 20 40 60 80 40 30


[ЭТАП 2]: Отображение построенного дерева (повернуто на 90°):


        80
    70
        60
50
            40
        40
    30
            30
        20


------------------------------------------------------------
[ЭТАП 3 - ЗАДАНИЕ 1]: Поиск элементов в дереве:
  Поиск ключа 40: НАЙДЕН (затрачено сравнений: 3).
  Поиск ключа 99: НЕ НАЙДЕН (затрачено сравнений: 3).


------------------------------------------------------------
[ЭТАП 4 - ЗАДАНИЕ 2]: Подсчет числа вхождений элементов:
  Число вхождений элемента 40: 2 раз(а)
  Число вхождений элемента 30: 2 раз(а)
  Число вхождений элемента 50: 1 раз(а)
  Число вхождений элемента 99: 0 раз(а)


------------------------------------------------------------
[ЭТАП 5 - ЗАДАНИЕ 3 (*)]: Добавление с фильтрацией дубликатов:
  Попытка добавить 25 (новое уникальное число):
  -> Узел 25 успешно добавлен в дерево.
  Попытка добавить 50 (уже существует в корне):
  [Исключение дубликата] Элемент 50 уже существует в дереве! Добавление отклонено.
  -> Дубликат 50 успешно отклонён.
  Попытка добавить 70 (уже существует в дереве):
  [Исключение дубликата] Элемент 70 уже существует в дереве! Добавление отклонено.
  -> Дубликат 70 успешно отклонён.


------------------------------------------------------------
[ЭТАП 6 - ЗАДАНИЕ 4 (*)]: Оценка сложности поиска в дереве:
============================================================
     АНАЛИТИЧЕСКАЯ ОЦЕНКА СЛОЖНОСТИ ПОИСКА В BST
============================================================
1. Лучший случай:  O(1)       — искомый элемент находится в корне.
2. Средний случай: O(log2 N)  — дерево приближено к сбалансированному.
                              При каждом шаге отсекается половина дерева.
3. Худший случай:  O(N)       — вырожденное дерево (список/лоза),
                              когда элементы добавлялись упорядоченно.
------------------------------------------------------------
Текущее состояние дерева:
  Количество узлов (N): 10
  Высота дерева (H):    4
  Теоретическая глубина сбалансированного: ~3.3 шагов
  Максимальное число сравнений сейчас:    4 шагов
============================================================
```

Анализ результатов тестирования:

1. Построение дерева и визуализация: тестовая последовательность чисел (50, 30, 70, 20, 40, 60, 80, 40, 30) была корректно организована в иерархическую структуру бинарного дерева поиска. Графический вывод с поворотом на 90 градусов подтверждает правильное распределение узлов: меньшие элементы ушли в левое поддерево (30, 20, 40), а большие — в правое (70, 60, 80).

2. Поиск элементов (Задание 1): при поиске существующего ключа 40 алгоритм нашел узел ровно за 3 сравнения (корень 50 -> влево на 30 -> вправо на 40). При поиске отсутствующего ключа 99 алгоритм совершил 3 сравнения и достиг указателя NULL, что подтверждает корректность завершения поиска.

3. Подсчет числа вхождений (Задание 2): для элементов 40 и 30 функция count_occurrences вернула значение 2, для уникального элемента 50 — значение 1, а для отсутствующего ключа 99 — значение 0. Это полностью соответствует составу тестовой последовательности.

4. Исключение дубликатов (Задание 3*): при попытке добавить новое уникальное число 25 элемент успешно включен в структуру. При попытке повторного добавления чисел 50 и 70 алгоритм insert_unique зафиксировал их наличие в дереве, выдал диагностическое сообщение и корректно отклонил операцию без нарушения структуры.

5. Оценка сложности (Задание 4*): для построенного дерева из N = 10 узлов фактическая глубина составила H = 4 уровня, что близко к теоретическому оптимуму сбалансированного дерева $\log_2(10) \approx 3.32$. Максимальное число шагов поиска не превысило 4, что подтверждает логарифмическую сложность $O(\log_2 N)$.

6. Выводы

- Изучены теоретические принципы организации динамических древовидных структур данных и особенности функционирования бинарных деревьев поиска (БДП).
- Разработана и программно реализована функция поиска элемента в дереве (search_tree), показавшая логарифмическую зависимость числа сравнений от общего количества узлов.
- Реализован алгоритм подсчета количества вхождений заданного значения (count_occurrences), безошибочно определяющий кратность элементов в дереве с дубликатами.

- Успешно решена задача добавления узлов с исключением дубликатов (insert_unique), обеспечивающая строгое хранение множества уникальных ключей и предотвращающая избыточный расход динамической памяти.
- Проведена аналитическая и практическая оценка сложности операций поиска, подтвердившая эффективность сбалансированного бинарного дерева поиска (O(log2 N)) по сравнению с линейным списком (O(N)).
- Разработанный программный комплекс на языке Си в среде Microsoft Visual Studio полностью выполнил все пункты лабораторного задания (включая задания повышенной сложности со звездочкой) и продемонстрировал надежное динамическое управление памятью без утечек.